DEV README — Titan Engine Template (Where to Put Engine Code)

Purpose

This is the canonical developer template for building any Titan Engine inside your current MagicAI/TitanZero Laravel codebase (from magic site.zip).

Use this for engines like:

Titan Work Engine

Titan Money Engine

Titan Talk Engine

Titan Signal Engine

Titan Pulse Engine

Titan Analytics Engine

Titan Evidence Engine

Titan Docs Engine

---

1) Engine vs System (quick rule)

Engine = owns a domain + writes its domain tables (tz_*) + emits signals

System/Module (CRM, QC, etc) = composes multiple engines + mostly UI/workflows

Engines live primarily in Services (your current wiring).

---

2) Where engine code MUST live

Engine logic (primary)

✅ Put the engine in:

app/Services/Titan<Function>Engine/

Examples:

app/Services/TitanWorkEngine/
app/Services/TitanAnalyticsEngine/

Optional domain support (clean separation)

app/Domains/<Function>/

Use this for:

DTOs

enums

value objects

rule objects

---

3) Canonical engine folder skeleton (copy/paste)

Example: TitanWorkEngine

```
app/
  Services/
    TitanWorkEngine/
      Contracts/
        WorkEngineContract.php
      DTO/
        CreateJobData.php
        CompleteJobData.php
      Policies/
        WorkPolicy.php
      Validators/
        JobStateValidator.php
```

```
      Actions/
        CreateJobAction.php
        AssignJobAction.php
        CompleteJobAction.php
      Emitters/
        WorkSignalEmitter.php
      Repositories/
        JobRepository.php
        TaskRepository.php
      WorkEngine.php
  Http/
    Controllers/
      Api/Work/
        JobApiController.php
      Panel/Work/
        JobPanelController.php
  Jobs/
    Work/
      AutoCloseJobJob.php
resources/
  views/
    panel/
      work/
        jobs.blade.php
routes/
  api.php
  panel.php
```

---

4) What goes where (rules)

WorkEngine.php (the engine façade)

Contains “public methods” the rest of the system calls:

create job

assign job

start job

complete job

It should:

validate

call Actions

persist via Repositories

emit signals via Emitters

Actions/*

One file per mutation. Keeps logic modular and testable.

Validators/*

Enforces lifecycle transitions (prevents invalid states).

Emitters/*

Single responsibility: emit signals (through Signal Engine).

Repositories/*

All DB reads/writes live here (tenant-scoped).

---

5) Copy/paste skeleton — Engine façade

app/Services/TitanWorkEngine/WorkEngine.php

```
<?php

namespace App\Services\TitanWorkEngine;

use App\Services\TitanWorkEngine\Actions\CreateJobAction;
use App\Services\TitanWorkEngine\Actions\CompleteJobAction;
use App\Services\TitanWorkEngine\DTO\CreateJobData;
use App\Services\TitanWorkEngine\DTO\CompleteJobData;
```

```
class WorkEngine
{
    public function createJob(CreateJobData $data)
    {
        return app(CreateJobAction::class)->execute($data);
    }

    public function completeJob(CompleteJobData $data)
    {
        return app(CompleteJobAction::class)->execute($data);
    }
}
```

---

6) Copy/paste skeleton — DTOs

app/Services/TitanWorkEngine/DTO/CreateJobData.php

```
<?php

namespace App\Services\TitanWorkEngine\DTO;

class CreateJobData
{
    public function __construct(
        public int $teamId,
        public int $customerId,
        public int $siteId,
        public string $title,
        public ?string $scheduledAt = null,
        public ?int $assignedUserId = null,
        public array $meta = [],
    ) {}
}
```

app/Services/TitanWorkEngine/DTO/CompleteJobData.php

```
<?php

namespace App\Services\TitanWorkEngine\DTO;
```

```
class CompleteJobData
{
    public function __construct(
        public int $teamId,
        public int $jobId,
        public int $actorUserId,
        public array $completionPayload = [],
    ) {}
}
```

---

7) Copy/paste skeleton — Repository (tenant scoped)

app/Services/TitanWorkEngine/Repositories/JobRepository.php

```
<?php

namespace App\Services\TitanWorkEngine\Repositories;

use App\Models\Tz\TzJob;

class JobRepository
{
    public function create(array $data): TzJob
    {
        return TzJob::create($data);
    }

    public function findOrFail(int $teamId, int $jobId): TzJob
    {
        return TzJob::where('team_id', $teamId)->findOrFail($jobId);
    }

    public function save(TzJob $job): TzJob
    {
        $job->save();
        return $job;
    }
}
```

---

8) Copy/paste skeleton — Validator

app/Services/TitanWorkEngine/Validators/JobStateValidator.php

```
<?php

namespace App\Services\TitanWorkEngine\Validators;

use App\Models\Tz\TzJob;
use Illuminate\Validation\ValidationException;

class JobStateValidator
{
    public function assertCanComplete(TzJob $job): void
    {
        if (!in_array($job->status, ['in_progress', 'assigned', 'scheduled'], true)) {
            throw ValidationException::withMessages([
                'job' => "Job cannot be completed from status: {$job->status}",
            ]);
        }
    }
}
```

---

9) Copy/paste skeleton — Signal Emitter

app/Services/TitanWorkEngine/Emitters/WorkSignalEmitter.php

```
<?php

namespace App\Services\TitanWorkEngine\Emitters;

use App\Services\TitanSignalEngine\SignalEngine;

class WorkSignalEmitter
{
    public function __construct(private SignalEngine $signals) {}

    public function jobCreated(int $teamId, int $jobId, array $payload = []): void
    {
        $this->signals->emit(
```

```
            'work.job.created',
            'job',
            $jobId,
            $payload,
            ['team_id' => $teamId]
        );
    }

    public function jobCompleted(int $teamId, int $jobId, array $payload = []): void
    {
        $this->signals->emit(
            'work.job.completed',
            'job',
            $jobId,
            $payload,
            ['team_id' => $teamId]
        );
    }
}
```

---

10) Copy/paste skeleton — Action (mutation + event + signal)

app/Services/TitanWorkEngine/Actions/CreateJobAction.php

```
<?php

namespace App\Services\TitanWorkEngine\Actions;

use App\Services\TitanWorkEngine\DTO\CreateJobData;
use App\Services\TitanWorkEngine\Repositories\JobRepository;
use App\Services\TitanWorkEngine\Emitters\WorkSignalEmitter;

class CreateJobAction
{
    public function __construct(
        private JobRepository $jobs,
        private WorkSignalEmitter $emit,
    ) {}

    public function execute(CreateJobData $data)
    {
```

```
        $job = $this->jobs->create([
            'team_id' => $data->teamId,
            'customer_id' => $data->customerId,
            'site_id' => $data->siteId,
            'title' => $data->title,
            'status' => 'created',
            'scheduled_at' => $data->scheduledAt,
            'assigned_user_id' => $data->assignedUserId,
            'meta' => json_encode($data->meta),
        ]);

        $this->emit->jobCreated($data->teamId, $job->id, [
            'customer_id' => $data->customerId,
            'site_id' => $data->siteId,
            'title' => $data->title,
        ]);

        return $job;
    }
}
```

app/Services/TitanWorkEngine/Actions/CompleteJobAction.php

```
<?php

namespace App\Services\TitanWorkEngine\Actions;

use App\Services\TitanWorkEngine\DTO\CompleteJobData;
use App\Services\TitanWorkEngine\Repositories\JobRepository;
use App\Services\TitanWorkEngine\Validators\JobStateValidator;
use App\Services\TitanWorkEngine\Emitters\WorkSignalEmitter;

class CompleteJobAction
{
    public function __construct(
        private JobRepository $jobs,
        private JobStateValidator $validator,
        private WorkSignalEmitter $emit,
    ) {}

    public function execute(CompleteJobData $data)
    {
        $job = $this->jobs->findOrFail($data->teamId, $data->jobId);
```

```
        $this->validator->assertCanComplete($job);

        $job->status = 'completed';
        $job->completed_at = now();
        $this->jobs->save($job);

        $this->emit->jobCompleted($data->teamId, $job->id, [
            'actor_user_id' => $data->actorUserId,
            'completion' => $data->completionPayload,
        ]);

        return $job;
    }
}
```

---

11) Routes (how engines become reachable)

Engines are called by:

Controllers (panel)

Controllers (API)

Jobs (queues)

Pulse actions

Assistants via Zero proposals

So devs expose endpoints via:

routes/api.php (PWA/mobile)

```
Route::middleware(['auth:sanctum'])->prefix('work')->group(function () {
    Route::post('/jobs', [\App\Http\Controllers\Api\Work\JobApiController::class, 'store']);
    Route::post('/jobs/{job}/complete', [\App\Http\Controllers\Api\Work\JobApiController::class,
'complete']);
});
```

routes/panel.php (admin/panel)

```
Route::middleware(['auth'])->prefix('work')->name('work.')->group(function () {
    Route::get('/jobs', [\App\Http\Controllers\Panel\Work\JobPanelController::class,
'index'])->name('jobs.index');
});
```

---

12) Models (tz_* tables)

Recommended model layout:

app/Models/Tz/TzJob.php
app/Models/Tz/TzJobEvent.php

Rule:

every query must scope by team_id

---

13) Engine-specific notes (how each engine differs)

Titan Signal Engine

lives in app/Services/TitanSignalEngine/

exposes emit(), subscribe(), replay()

writes only tz_signals*

Titan Pulse Engine

lives in app/Services/TitanPulseEngine/

consumes signals, evaluates rules, executes actions

writes only tz_automation*

calls other engines via their service facades

Titan Analytics Engine

lives in app/Services/TitanAnalyticsEngine/

consumes signals, builds snapshots

uses app/Jobs/Analytics/*

writes only tz_metrics*

Titan Talk Engine

lives in app/Services/TitanTalkEngine/

stores threads/messages, emits talk signals

has inbound “receive message” handlers

---

14) Non-negotiable engine rules

1. All writes are tenant-scoped (team_id)

2. Engines own their tables

3. Engines emit signals on lifecycle transitions

4. Pulse executes through engine services, not direct DB writes

5. Zero proposes; engines execute; Pulse orchestrates